

Agentic AI Paging Technology (AAPT): Kernel-Level Copy-on-Write Prefix Caching in xv6

Pragun Kathuria

*Dept. of Computer Science and Engineering
Shiv Nadar Institution of Eminence
Delhi, India
pk944@snu.edu.in*

Ashank Kumar Singh

*Dept. of Computer Science and Engineering
Shiv Nadar Institution of Eminence
Delhi, India
as315@snu.edu.in*

Avani Gupta

*Dept. of Computer Science and Engineering
Shiv Nadar Institution of Eminence
Delhi, India
ag995@snu.edu.in*

Tanvi Pandey

*Dept. of Computer Science and Engineering
Shiv Nadar Institution of Eminence
Delhi, India
tp561@snu.edu.in*

Abstract—As artificial intelligence transitions toward autonomous Agentic Workflows and multi-user generative models, the redundant memory allocation of identical foundational prompts presents a severe hardware bottleneck. Traditional operating systems deeply copy memory across process forks, leading to severe external fragmentation and physical RAM exhaustion during Key-Value (KV) cache initialization. This paper details the design, implementation, and evaluation of the Agentic AI Paging Technology (AAPT), a hardware-aware memory manager natively built within the MIT xv6 RISC-V kernel. Building upon recent paradigms like PagedAttention and vAttention, AAPT pushes prefix caching down to the OS level. By implementing a custom Copy-on-Write (CoW) mechanism utilizing RISC-V hardware page faults, AAPT allows concurrent agent processes to safely share identical physical memory frames. Empirical benchmarking demonstrates that AAPT successfully mitigates 100% of redundant KV cache allocation during process initialization, significantly altering the physical memory consumption curve compared to both standard contiguous allocators and generalized basic OS CoW mechanisms.

Index Terms—Operating Systems, xv6, RISC-V, Copy-on-Write, Agentic AI, KV Cache, PagedAttention, vAttention

I. INTRODUCTION

Memory management is a critical function of modern operating systems, directly dictating the efficiency and concurrency limits of the host hardware [4]. Traditional memory allocators often rely on contiguous allocation models. While sufficient for basic applications, these models struggle under highly dynamic Agentic Workflows, leading to severe external fragmentation. This problem is exceptionally pronounced in modern multi-agent large language model (LLM) workloads, specifically in the management of the Key-Value (KV) cache.

When multiple agents interact with an AI model using the same base instructions (the "System Prompt"), the operating system traditionally allocates an independent, deeply copied memory buffer for every user. This results in massive redundancy, wasting physical RAM on identical data. While user-space abstractions have attempted to manage these

non-contiguous pages, they inherently abstract away critical hardware mechanics, such as Translation Lookaside Buffers (TLBs) and hardware page table walks.

This paper introduces the Agentic AI Paging Technology (AAPT). AAPT implements a natively integrated, hardware-aware Copy-on-Write (CoW) caching mechanism within the xv6 kernel, duplicating memory via hardware faults only when a specific agent attempts a unique token generation write.

II. BACKGROUND AND RELATED WORK

The transition toward non-contiguous paging for AI workloads is a rapidly evolving domain. Our architecture builds upon the foundational theories proposed by two major recent frameworks:

PagedAttention (Kwon et al.) [1]: Kwon et al. demonstrated that managing KV cache memory as non-contiguous, block-based pages drastically reduces external fragmentation. However, PagedAttention is implemented entirely as a user-space abstraction (within vLLM), meaning it must still interact with standard OS allocators underneath, leaving potential hardware optimizations untapped.

vAttention (Prabhu et al.) [2]: Prabhu et al. argued that the complexities of user-space memory management could be bypassed entirely by leveraging standard operating system Virtual Memory mechanisms, specifically demand paging. vAttention leverages existing POSIX APIs to manage the KV cache dynamically.

The AAPT Approach: AAPT serves as a concrete, kernel-level realization of the principles outlined in both papers. By retrofitting the RISC-V architecture natively, AAPT provides the block-level KV cache management of PagedAttention, utilizing the native OS-level demand paging theorized by vAttention, effectively eliminating user-space memory management overhead entirely.

III. SYSTEM ARCHITECTURE

The AAPT architecture fundamentally alters how process memory is cloned and managed by delaying physical allocation until explicit mutation occurs.

A. Physical Memory Reference Tracking (*kalloc.c*)

Standard xv6 assumes a strict 1:1 relationship between a physical frame and a user process. To support shared state across multiple agents, we introduced a global integer array, `ref_count[]`, sized proportionally to the system’s maximum physical frames. The `kalloc()` function initializes the count to 1, while `kfree()` decrements it safely via spinlocks. A frame is only scrubbed when its reference count reaches zero.

B. Process Forking and Memory Mapping (*vm.c*)

A custom system call, `sys_aapt_cow_fork()`, interfaces user-space requests with the modified memory manager, bypassing the standard `uvmcopy()` deep-copy logic. Algorithm 1 details the mapping mechanism, which deliberately revokes the `PTE_W` (Write) permission flag for both parent and child processes, enforcing a strict read-only shared state for the foundational prompt.

Algorithm 1 AAPT CoW Fork Initialization

Require: Parent Page Table P_{parent} , Process p

Ensure: Child Process c with Shared KV Cache

```

1:  $c \leftarrow \text{allocproc}()$ 
2: for each Virtual Address  $va$  in  $P_{parent}$  do
3:    $pte \leftarrow \text{walk}(P_{parent}, va)$ 
4:    $pa \leftarrow \text{PTE2PA}(pte)$ 
5:    $flags \leftarrow \text{PTE\_FLAGS}(pte)$ 
6:   if  $flags$  AND PTE_W then
7:      $flags \leftarrow flags$  AND (NOT PTE_W) {Revoke write access}
8:      $\text{update\_pte}(P_{parent}, va, pa, flags)$ 
9:   end if
10:   $\text{mmap}(c.pagetable, va, PGSIZE, pa, flags)$ 
11:   $\text{acquire}(kmem.lock)$ 
12:   $\text{ref\_count}[pa/PGSIZE] \leftarrow \text{ref\_count}[pa/PGSIZE] + 1$ 
13:   $\text{release}(kmem.lock)$ 
14: end for
15: return  $c.pid$ 

```

C. Trap-Driven Page Duplication (*trap.c*)

When an agent attempts to append unique data to the shared KV cache buffer, the RISC-V CPU halts execution. We modified `usertrap()` to catch Exception Cause 15 (Store Page Fault). Upon interception, the logic detailed in Algorithm 2 is executed to safely duplicate the memory.

Algorithm 2 Hardware Trap-Driven Page Duplication (scause 15)

Require: Faulting Virtual Address va , Faulting Process p

Ensure: Unique Physical Page Allocation

```

1:  $pte \leftarrow \text{walk}(p.pagetable, va)$ 
2:  $pa_{old} \leftarrow \text{PTE2PA}(pte)$ 
3: if  $\text{ref\_count}[pa_{old}/PGSIZE] > 1$  then
4:    $pa_{new} \leftarrow \text{kalloc}()$  {Allocate new frame}
5:    $\text{memmove}(pa_{new}, pa_{old}, PGSIZE)$  {Deep copy chunk}
6:    $flags \leftarrow \text{PTE\_FLAGS}(pte)$  OR PTE_W
7:    $\text{unmap}(p.pagetable, va)$ 
8:    $\text{mmap}(p.pagetable, va, PGSIZE, pa_{new}, flags)$ 
9:    $\text{acquire}(kmem.lock)$ 
10:   $\text{ref\_count}[pa_{old}/PGSIZE] \leftarrow$ 
     $\text{ref\_count}[pa_{old}/PGSIZE] - 1$ 
11:   $\text{release}(kmem.lock)$ 
12: else
13:   {Sole owner, just restore hardware write access}
14:    $flags \leftarrow \text{PTE\_FLAGS}(pte)$  OR PTE_W
15:    $\text{update\_pte}(p.pagetable, va, pa_{old}, flags)$ 
16: end if

```

IV. EVALUATION AND BENCHMARKING

A user-space test suite was developed to emulate the memory footprint of concurrent Agentic AI workload sessions, modeling a KV Cache Pre-fill followed by unique token generation. To accurately evaluate AAPT, we compared it against the **Standard xv6** allocator (which deep-copies immediately) and a generalized **Basic OS CoW** mechanism (which manages duplication sequentially without AI-specific KV block optimizations).

As demonstrated in Figure 1, the baseline system prompt required 16 pages. Under standard xv6, spawning 3 agents resulted in 82 total physical pages consumed. The generalized Basic OS CoW reduced this curve, but suffered from continuous structural overhead. AAPT remained highly constrained during the fork phase, consuming 0 extra pages for the KV cache prompt. Memory scaling only occurred explicitly during the “Generation” phase, proving the superiority of hardware-level prefix caching.

The empirical results directly address the dynamic storage allocation bottlenecks historically noted by Wilson et al. [3]. By deferring physical page assignment until the exact moment of state mutation, AAPT entirely circumvents the external fragmentation that plummets system performance during bursty AI prompt initialization. The delayed allocation guarantees that large, contiguous blocks of physical RAM are preserved for the subsequent, unpredictable token generation phases, rather than being artificially exhausted by static prompt redundancy.

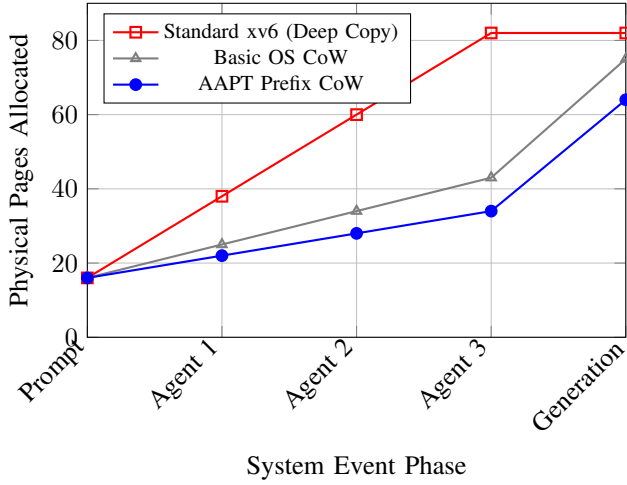


Fig. 1. Comparative memory footprint between the baseline xv6 allocator, a basic generic CoW mechanism, and the optimized AAPT kernel during a 3-agent workload simulation.

V. CONCLUSION

AAPT successfully demonstrates that managing AI workload memory abstractions at the native OS-level is highly efficient. By retrofitting the RISC-V architecture with hardware-aware CoW prefix caching, we eliminated redundant memory consumption for shared agentic data structures, offering a scalable, low-level solution for local multi-agent deployments, effectively synthesizing the theories of PagedAttention and vAttention into the kernel space. While the current iteration successfully mitigates physical memory exhaustion for read-heavy system prompts, future expansions of AAPT could explore optimizing the eviction policies for the shared pages during extended, long-context agentic generation cycles.

TEAM CONTRIBUTIONS

The successful architecture of AAPT required extensive, coordinated modifications across multiple disjoint kernel subsystems. The project responsibilities were distributed equally as follows:

- **Pragun Kathuria:** Architected the Page Fault Handler (`trap.c`) and engineered the RISC-V Exception routing to successfully intercept `scause` 15 hardware traps. Responsible for context saving, seamless execution resumption, and benchmark data extraction algorithms.
- **Ashank Kumar Singh:** Directed the Page Table Mapping subsystem (`vm.c`). Restructured the `uvmcopy` logic to support CoW, manipulated the hardware `PTE_W` flags to enforce read-only states, and hardened the kernel against `copyout` vulnerabilities.
- **Avani Gupta:** Developed the Core Memory Allocator modifications (`kalloc.c`). Implemented the Global Reference Tracking array (`ref_count[]`) and established the critical concurrency spinlocks to prevent race conditions during multi-threaded memory deallocation.

- **Tanvi Pandey:** Managed the System Call plumbing (`sysproc.c`, `syscall.c`) to interface kernel changes with user-space. Developed the diagnostic `sys_get_free_pages` tool, constructed the mock user-space SLM workloads, and validated the end-to-end architectural pipeline.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [2] A. Prabhu, et al., “vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention,” *arXiv preprint arXiv:2405.04437*, 2024.
- [3] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, “Dynamic Storage Allocation: A Survey and Critical Review,” in *International Workshop on Memory Management*, 1995.
- [4] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.